Web Scraping for Applied Research

Why scrape

- Some information is public, but exists only as web pages.
- Job postings state salaries. No standard dataset collects them.
- An afternoon of scraping produces a usable dataset.
- The same method works for courts, procurement, company registers.

The example

- Several US states now require postings to state a salary range.
- Some firms quote a different range for each city.
- Same firm, same job, same text. Only the city differs.
- So: how much does the posted range change with location?

The answer, collected today

- 525 postings quote more than one city.
- The average gap between top and bottom city is 22%.
- For the typical posting it is 15%.
- Collecting the data took about ten seconds.

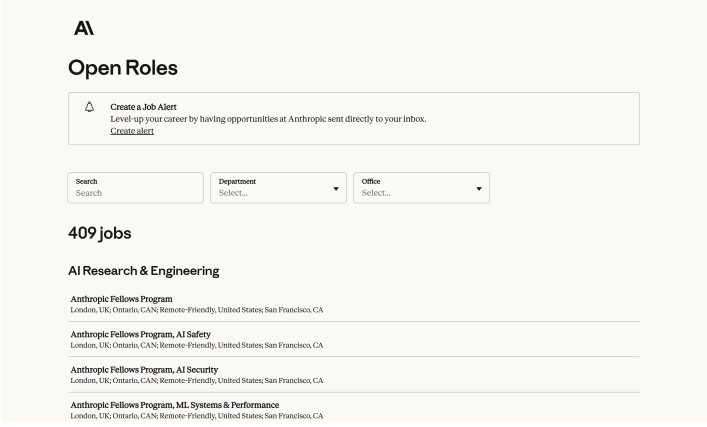
Plan

- Look at the page before writing any code.
- How the web actually works.
- Three ways to collect the same numbers.
- From three postings to nine thousand.
- The price of a job attribute: location, skills, an amenity.
- Doing this responsibly.

Rule number one

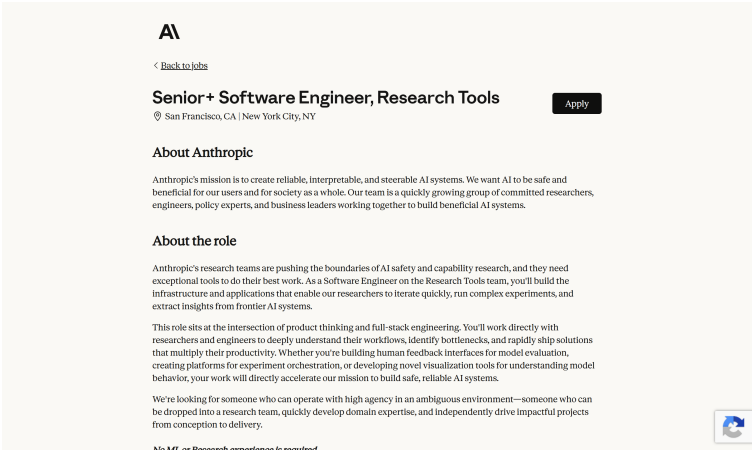
- Never write a scraper for a page you have not read.
- Find the number on screen first.
- Then ask how the page obtained it.
- Ten minutes of looking saves an afternoon of guessing.

The board: every job at one firm



A list of links. No salaries here.

One posting



Back to jobs

Senior+ Software Engineer, Research Tools

Apply

San Francisco, CA | New York City, NY

About Anthropic

Anthropic's mission is to create reliable, interpretable, and steerable AI systems. We want AI to be safe and beneficial for our users and for society as a whole. Our team is a quickly growing group of committed researchers, engineers, policy experts, and business leaders working together to build beneficial AI systems.

About the role

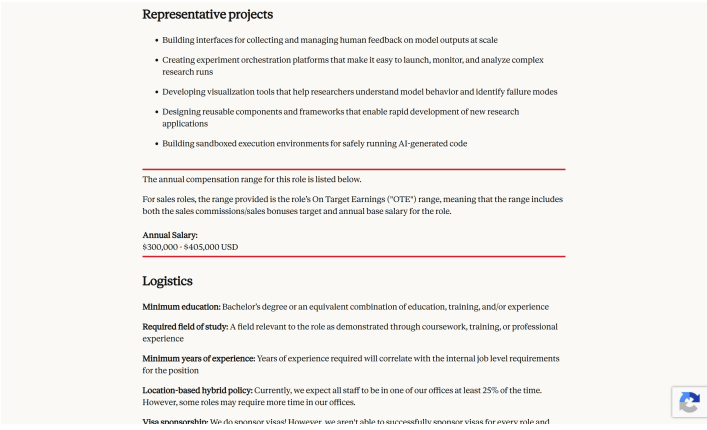
Anthropic's research teams are pushing the boundaries of AI safety and capability research, and they need exceptional tools to do their best work. As a Software Engineer on the Research Tools team, you'll build the infrastructure and applications that enable our researchers to iterate quickly, run complex experiments, and extract insights from frontier AI systems.

This role sits at the intersection of product thinking and full-stack engineering. You'll work directly with researchers and engineers to deeply understand their workflows, identify bottlenecks, and rapidly ship solutions that multiply their productivity. Whether you're building human feedback interfaces for model evaluation, creating platforms for experiment orchestration, or developing novel visualization tools for understanding model behavior, your work will directly accelerate our mission to build safe, reliable AI systems.

We're looking for someone who can operate with high agency in an ambiguous environment—someone who can be dropped into a research team, quickly develop domain expertise, and independently drive impactful projects from conception to delivery.

No ML or Research experience is required

Scroll down: the numbers



Representative projects

- Building interfaces for collecting and managing human feedback on model outputs at scale
- Creating experiment orchestration platforms that make it easy to launch, monitor, and analyze complex research runs
- Developing visualization tools that help researchers understand model behavior and identify failure modes
- Designing reusable components and frameworks that enable rapid development of new research applications
- Building sandboxed execution environments for safely running AI-generated code

The annual compensation range for this role is listed below.

For sales roles, the range provided is the role's On Target Earnings ("OTE") range, meaning that the range includes both the sales commissions/sales bonuses target and annual base salary for the role.

Annual Salary:
$300,000 - $405,000 USD

Logistics

Minimum education: Bachelor's degree or an equivalent combination of education, training, and/or experience

Required field of study: A field relevant to the role as demonstrated through coursework, training, or professional experience

Minimum years of experience: Years of experience required will correlate with the internal job level requirements for the position

Location-based hybrid policy: Currently, we expect all staff to be in one of our offices at least 25% of the time. However, some roles may require more time in our offices.

Visa sponsorship: We do sponsor visas! However, we aren't able to successfully sponsor visas for every role and

Postings change constantly, so your numbers will differ from these.

Where the numbers sit

```
<div class="pay-input">
  <bdi>$300,000</bdi>
  <span>-</span>
  <bdi>$405,000</bdi>
  <bdi>USD</bdi>
</div>
```

- Right-click the salary, choose Inspect.
- The numbers sit inside `<bdi>` tags.
- That tag name is all we need to find them again.

The page asks for its own data

- A page loads, then requests more things from the server.
- Press F12, open Network, filter to Fetch/XHR, reload.
- One of those requests returns the job as data.
- Open that address yourself and skip the page entirely.

The same job, as data

```
{
  "title": "Senior+ Software Engineer, Research Tools",
  "location": { "name": "San Francisco, CA | New York City, NY" },
  "pay_input_ranges": [
    { "min_cents": 30000000,
      "max_cents": 40500000,
      "currency_type": "USD" }
  ]
}
```

- The salary is already a number, in cents.
- The currency is stated explicitly.
- Nothing needs to be parsed out of text.

Requests and replies

- You send a request to an address.
- The server replies with a status code and some content.
- Scraping is doing by hand what your browser does automatically.
- Always check the code: a 404 page parses into nonsense.

Status codes worth knowing

- **200** — fine, here it is.
- **403** — I know who you are, and no.
- **404** — no such thing.
- **429** — you are asking too fast.
- **500** — the server broke.

Two kinds of reply

HTML

```
<div class="pay-input">
  <bdi>$300,000</bdi>
  <span>-</span>
  <bdi>$405,000</bdi>
</div>
```

JSON

```
{
  "min_cents": 30000000,
  "max_cents": 40500000,
  "currency_type": "USD"
}
```

- HTML mixes content with layout, and changes when the design changes.
- JSON is names and values, and becomes a Python dictionary in one line.

What to try, in order

- Is there a bulk download? Take it and stop.
- Is there an official API? Use it.
- Is there a hidden JSON address? Check the Network tab.
- Is the content in the HTML? Use requests and BeautifulSoup.
- Does it need a real browser? Use Selenium.
- Are you being blocked? Change the question, not the tooling.

Method 1: request the data directly

```
import requests

url = ("https://boards-api.greenhouse.io/v1/boards/anthropic"
       "/jobs/4981828008?pay_transparency=true")

r = requests.get(url, timeout=60)
r.raise_for_status()

job = r.json()
pay = job["pay_input_ranges"][0]
```

- `.json()` returns a dictionary; we read fields from it.
- `pay["min_cents"] / 100` gives 300000.0.

Method 2: parse the HTML

```
from bs4 import BeautifulSoup

html = requests.get(url_web, timeout=60).text
soup = BeautifulSoup(html, "html.parser")

texts = [t.get_text(strip=True) for t in soup.find_all("bdi")]
# ['$300,000', '$405,000', 'USD']
```

- We must strip the dollar signs and commas ourselves.
- It works only because we looked and found `<bdi>`.
- If the design changes, this breaks without warning.

Method 3: drive a real browser

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--headless=new")
driver = webdriver.Chrome(options=options)

driver.get(url_web)
WebDriverWait(driver, 20).until(
    EC.presence_of_element_located((By.TAG_NAME, "bdi")))
```

- Use it when JavaScript builds the page, or you must click.
- Always wait for the element; never assume the page is ready.
- Always close the browser when finished.

The error you will meet first

```
StaleElementReferenceException: stale element reference
```

- You found an element, then the page rebuilt itself.
- Your variable now points at something that no longer exists.
- The fix is to wait, then look the element up again.
- This is not your bug; it is what browsers do.

All three give the same numbers

Posting	Lower	Upper	Width
Senior+, Research Tools	\$300,000	\$405,000	29.8%
Staff, Labs: Applied AI	\$320,000	\$405,000	23.4%
Staff+, Platform	\$405,000	\$485,000	18.0%

- The method changes the cost and the fragility, not the answer.

What each method costs

Method	Total (3)	Per posting	Relative
JSON endpoint	1.4 s	0.45 s	1.0×
requests + BeautifulSoup	1.9 s	0.64 s	1.4×
Selenium	2.1 s	0.71 s	1.6×

- Small differences at three postings.
- They matter once you want thousands.

Ask once for everything

Approach	Time for one firm (about 400 postings)	Relative
Board endpoint, one request	1 s	1×
JSON, one posting at a time	3 min	170×
BeautifulSoup, per page	4–5 min	235×
Selenium, per page	5 min	260×

- Almost all the time is network round-trips, not parsing.
- Fewer requests is the only optimisation that matters.

Four new problems at scale

- Things fail: boards close, names change, connections drop.
- Waiting dominates: run several requests at once.
- Re-running wastes everything: cache each reply to disk.
- You are a guest: identify yourself and stay modest.

All four, in twenty lines

```
def fetch_board(company, use_cache=True):
    path = f"data/cache/{company}.json"
    if use_cache and os.path.exists(path):
        return json.load(open(path))

    try:
        r = SESSION.get(BOARD_API.format(company=company), timeout=45)
        if r.status_code != 200:
            return []
        jobs = r.json().get("jobs", [])
    except Exception:
        return []

    json.dump(jobs, open(path, "w"))
    return jobs

with ThreadPoolExecutor(max_workers=8) as pool:
    results = pool.map(fetch_board, companies)
```

What came back

Firm boards requested		109
returned data		63
Postings collected		8,852
with a salary range	4,022	(45%)
quoting several cities		583

- Many boards returned nothing, which is normal.
- Fewer than half disclose pay, mostly where the law requires it.

A job is a bundle of attributes

- A job is a bundle: a place, some skills, an office or not.
- The wage is the price of the whole bundle. (Rosen, 1974)
- Regress pay on the attributes and you get the price of each.
- The posting shows the attributes and the price, so we can.

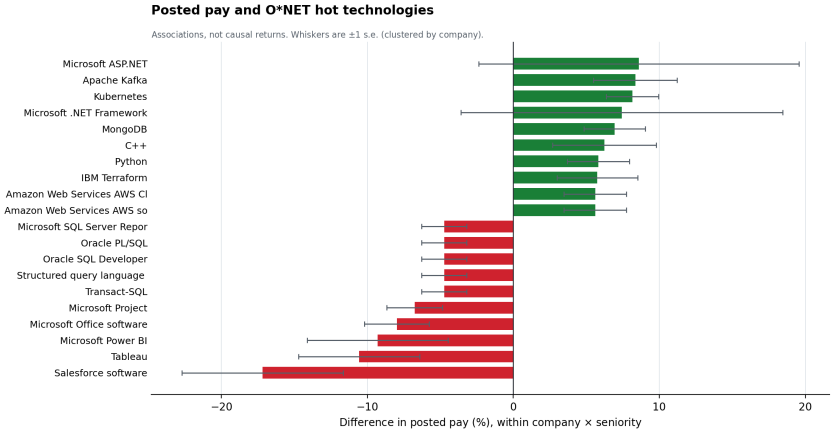
We do this for three attributes: location, skills, and one amenity.

1. The price of location

Postings quoting more than one city	525
Gap between top and bottom city, mean	22.4%
median	15.4%
90th percentile	28.7%

- Same posting, same text — only the city changes.
- So the gap is the price of location, nothing else.

2. The price of skills



A hedonic regression: each bar is one skill's price, within firm and seniority.

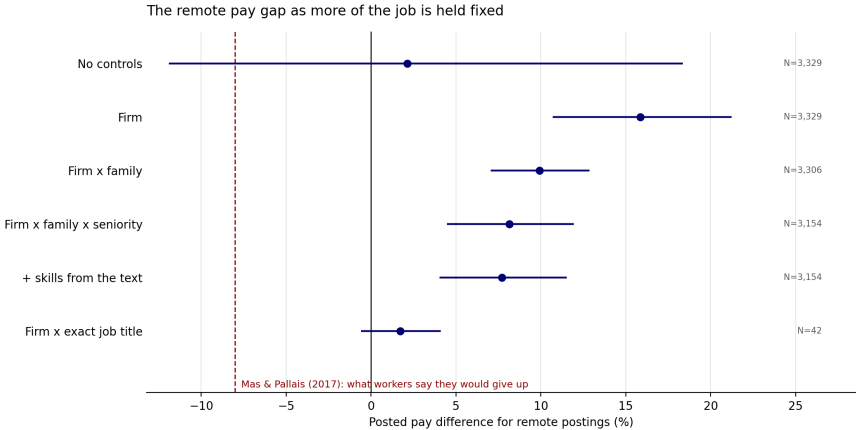
2. The price of skills

- This is a hedonic regression: pay on skill dummies.
- Data and infrastructure skills are priced highest.
- Office and reporting tools are priced lowest.
- A market price, not a causal return: Excel marks admin roles.

3. The price of an amenity: remote work

- Remote work is an amenity: workers value it directly.
- For an amenity, the price should be negative — you pay for it.
- That negative price is the compensating differential.
- Mas and Pallais (2017) estimate it near -8%.

3. The price of an amenity: remote work



3. The price of an amenity: remote work

- Raw, remote pays more: remote jobs are better jobs.
- Hold the job fixed and the price falls to about zero.
- So no compensating differential in posted pay here.
- The tightest comparison rests on 42 postings.

Scraping responsibly

- Prefer a bulk download or an official API when one exists.
- Send a User-Agent that says who you are.
- Ask for many records per request, not many requests.
- Slow down when you see 429; read robots.txt and the terms.
- Do not collect personal data you do not need.
- Keep the raw replies, so results can be reproduced.

Your turn

- Add ten firms of your choice and re-run the scrape.
- Pick a feature the postings mention: seniority, department, location.
- Ask whether posted pay differs along it, within the same firm.
- Write one paragraph on what you found and why it might be wrong.

What to remember

- Look at the page before writing code.
- Check for a hidden JSON address before parsing HTML.
- Make fewer requests, not faster parsers.
- Expect failure, and cache every raw reply.
- Collecting the data is the easy part.
- What you compare is the research.

Appendix

Compensating differentials

- Theory: bad job attributes should pay more, good ones less.
- Wage data struggles because better workers get both.
- A posted range is attached to a vacancy, not to a worker.
- So worker selection does not enter the observation directly.

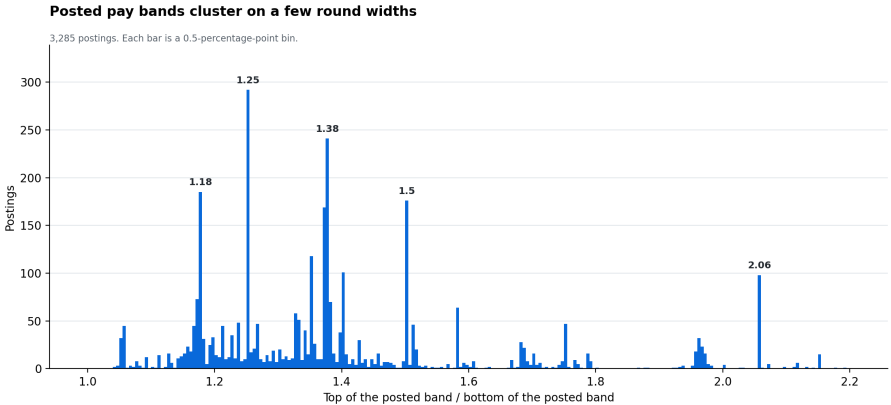
How the skills were extracted

- The list is external: O*NET Hot Technologies, 170 items.
- Choosing terms after seeing the results would be circular.
- Only requirement sections are read, not company blurb.
- Negation is handled, as in this real sentence:

```
"No ML or Research experience is required"
```

- Words like access, cloud and team need the full product name.

A separate question: how is the band set?



Not about attributes and pay — about how the firm chooses the range.

Reading the width result

- A band could mean room to negotiate, or a role spanning levels.
- It could also be a convention that satisfies a disclosure rule.
- Spikes at round ratios point to convention.
- But a flat percentage is not a flat amount.
- 31% of $400k is $124k; 31% of $130k is $40k.
- So this says nothing about whether people negotiate.

What this sample is

- Firms using one hiring platform, mostly technology.
- Only postings that disclose a range.
- Advertised pay, not paid pay.
- Not the labour market, and not a random sample of it.

The false start

```
pat = re.compile(r"\$\s?(\d{1,3}(?:,\d{3})+)\s*-\s*\$\s?(\d{1,3}(?:,\d{3})+)")
```

- Searching the text with a regular expression found 13% of salaries.
- Adding `?pay_transparency=true` found 45%, as typed numbers.
- The missing cases were not in the text at all.
- Look for structure before writing a regular expression.

Politeness in code

```
def get(url, tries=4):
    for attempt in range(tries):
        r = SESSION.get(url, timeout=45)
        if r.status_code == 429:
            time.sleep(2 ** attempt)
            continue
        if r.status_code >= 500:
            time.sleep(1 + attempt)
            continue
        return r
    return None
```

- Wait 1s, then 2s, then 4s before retrying.
- Hammering a struggling server is how you get blocked.

Reuse the connection

```
for url in urls:
    requests.get(url)
```

```
s = requests.Session()
for url in urls:
    s.get(url)
```

- Each new connection re-negotiates encryption with the server.
- A session reuses it, and remembers headers and cookies.
- Without this, BeautifulSoup looked slower than Selenium.

Common errors

Error	Usually means
`ModuleNotFoundError`	installed for a different Python
`JSONDecodeError`	the reply was not JSON; print `r.text`
`'NoneType' has no...`	`soup.find()` found nothing; the tag changed
`StaleElementReference`	the page rebuilt itself; wait and look again
`SessionNotCreated`	Selenium older than 4.25, or no Chrome
HTTP 403	the server refused you; check the terms
HTTP 429	you are going too fast

Where to go next

- `playwright`: a faster, more modern Selenium.
- `scrapy`: a framework for large crawls.
- Government portals: procurement, courts, transparency registers.
- The Internet Archive turns a static site into a historical panel.
- Open a page, press F12, and watch the Network tab.